Chapter 9. ReplicaSets

We have covered how to run individual containers as Pods, but these Pods are essentially one-off singletons. More often than not, you want multiple replicas of a container running at a particular time for a variety of reasons:

Redundancy

: Failure toleration by running multiple instances.

Scale

: Higher request-processing capacity by running multiple instances.

Sharding

: Different replicas can handle different parts of a computation in parallel.

Of course, you could manually create multiple copies of a Pod using multiple different (though largely similar) Pod manifests, but doing so is both tedious and error-prone. Logically, a user managing a replicated set of Pods considers them as a single entity to be defined and managed—and that's precisely what a ReplicaSet is. A ReplicaSet acts as a cluster-wide Pod manager, ensuring that the right types and numbers of Pods are running at all times.

Because ReplicaSets make it easy to create and manage replicated sets of Pods, they are the building blocks for common application deployment patterns and for self-healing applications at the infrastructure level. Pods managed by ReplicaSets are automatically rescheduled under certain failure conditions, such as node failures and network partitions.

The easiest way to think of a ReplicaSet is that it combines a cookie cutter and a desired number of cookies into a single API object. When we define a ReplicaSet, we define a specification for the Pods we want to create (the "cookie cutter") and a desired number of replicas. Additionally, we need to define a way of finding Pods that the ReplicaSet should control. The actual act of managing the replicated Pods is an example of a *recon-*

ciliation loop. Such loops are fundamental to most of the design and implementation of Kubernetes.

Reconciliation Loops

The central concept behind a reconciliation loop is the notion of *desired* state versus *observed* or *current* state. Desired state is the state you want. With a ReplicaSet, it is the desired number of replicas and the definition of the Pod to replicate. For example, "the desired state is that there are three replicas of a Pod running the `kuard` server." In contrast, the current state is the currently observed state of the system. For example, "there are only two `kuard` Pods currently running."

The reconciliation loop is constantly running, observing the current state of the world and taking action to try to make the observed state match the desired state. For instance, with the previous examples, the reconciliation loop would create a new `kuard` Pod in an effort to make the observed state match the desired state of three replicas.

There are many benefits to the reconciliation-loop approach to managing state. It is an inherently goal-driven, self-healing system, yet it can often be easily expressed in a few lines of code. For example, the reconciliation loop for ReplicaSets is a single loop, yet it handles user actions to scale up or scale down the ReplicaSet as well as node failures or nodes rejoining the cluster after being absent.

We'll see numerous examples of reconciliation loops in action throughout the rest of the book.

Relating Pods and ReplicaSets

Decoupling is a key theme in Kubernetes. In particular, it's important that all of the core concepts of Kubernetes are modular with respect to each other and that they are swappable and replaceable with other components. In this spirit, the relationship between ReplicaSets and Pods is loosely coupled. Though ReplicaSets create and manage Pods, they do not own the Pods they create. ReplicaSets use label queries to identify the set of Pods they should be managing. They then use the exact same Pod API that you used directly in Chapter 5 to create the Pods that they are managing. This notion of "coming in the front door" is another central design concept in Kubernetes. In a similar decoupling, ReplicaSets that create multiple Pods and the services that load balance to those Pods are also totally separate, decoupled API objects. In addition to supporting modularity,

decoupling Pods and ReplicaSets enables several important behaviors, discussed in the following sections.

Adopting Existing Containers

Although declarative configuration is valuable, there are times when it is easier to build something up imperatively. In particular, early on you may be simply deploying a single Pod with a container image without a ReplicaSet managing it. You might even define a load balancer to serve traffic to that single Pod.

But at some point, you may want to expand your singleton container into a replicated service and create and manage an array of similar containers. If ReplicaSets owned the Pods they created, then the only way to start replicating your Pod would be to delete it and relaunch it via a ReplicaSet. This might be disruptive, as there would be a moment when there would be no copies of your container running. However, because ReplicaSets are decoupled from the Pods they manage, you can simply create a ReplicaSet that will "adopt" the existing Pod and scale out additional copies of those containers. In this way, you can seamlessly move from a single imperative Pod to a replicated set of Pods managed by a ReplicaSet.

Quarantining Containers

Oftentimes, when a server misbehaves, Pod-level health checks will automatically restart that Pod. But if your health checks are incomplete, a Pod can be misbehaving but still be part of the replicated set. In these situations, while it would work to simply kill the Pod, that would leave your developers with only logs to debug the problem. Instead, you can modify the set of labels on the sick Pod. Doing so will disassociate it from the ReplicaSet (and service) so that you can debug the Pod. The ReplicaSet controller will notice that a Pod is missing and create a new copy, but because the Pod is still running, it is available to developers for interactive debugging, which is significantly more valuable than debugging from logs.

Designing with ReplicaSets

ReplicaSets are designed to represent a single, scalable microservice inside your architecture. Their key characteristic is that every Pod the ReplicaSet controller creates is entirely homogeneous. Typically, these Pods are then fronted by a Kubernetes service load balancer, which spreads traffic across the Pods that make up the service. Generally speaking, ReplicaSets are designed for stateless (or nearly stateless) services. The ele-

ments they create are interchangeable; when a ReplicaSet is scaled down, an arbitrary Pod is selected for deletion. Your application's behavior shouldn't change because of such a scale-down operation.

NOTE

Typically you will see applications use the Deployment object because it allows you to manage the release of new versions. ReplicaSets power Deployments under the hood, and it's important to understand how they operate so that you can debug them should you need to troubleshoot.

ReplicaSet Spec

Like all objects in Kubernetes, ReplicaSets are defined using a specification. All ReplicaSets must have a unique name (defined using the `metadata.name` field), a `spec` section that describes the number of Pods (replicas) that should be running cluster-wide at any given time, and a Pod template that describes the Pod to be created when the defined number of replicas is not met. Example 9-1 shows a minimal ReplicaSet definition. Pay attention to the replicas, selector, and template sections of the definition because they provide more insight into how ReplicaSets operate.

Example 9-1. kuard-rs.yaml

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  labels:
    app: kuard
    version: "2"
  name: kuard
spec:
  replicas: 1
  selector:
    matchLabels:
      app: kuard
      version: "2"
  template:
    metadata:
      labels:
        app: kuard
        version: "2"
    spec:
```

```
    containers:
      - name: kuard
        image: "gcr.io/kuar-demo/kuard-amd64:green"
```

Pod Templates

As mentioned previously, when the number of Pods in the current state is less than the number of Pods in the desired state, the ReplicaSet controller will create new Pods using a template contained in the ReplicaSet specification. The Pods are created in exactly the same manner as when you created a Pod from a YAML file in previous chapters, but instead of using a file, the Kubernetes ReplicaSet controller creates and submits a Pod manifest based on the Pod template directly to the API server. Here is an example of a Pod template in a ReplicaSet:

```
template:
  metadata:
    labels:
      app: helloworld
      version: v1
  spec:
    containers:
      - name: helloworld
        image: kelseyhightower/helloworld:v1
        ports:
          - containerPort: 80
```

Labels

In any reasonably sized cluster, many different Pods are running simultaneously—so how does the ReplicaSet reconciliation loop discover the set of Pods for a particular ReplicaSet? ReplicaSets monitor cluster state using a set of Pod labels to filter Pod listings and track Pods running within a cluster. When initially created, a ReplicaSet fetches a Pod listing from the Kubernetes API and filters the results by labels. Based on the number of Pods the query returns, the ReplicaSet deletes or creates Pods to meet the desired number of replicas. These filtering labels are defined in the ReplicaSet `spec` section and are the key to understanding how ReplicaSets work.

NOTE

The selector in the ReplicaSet `spec` should be a proper subset of the labels in the Pod template.

Creating a ReplicaSet

ReplicaSets are created by submitting a ReplicaSet object to the Kubernetes API. In this section, we will create a ReplicaSet using a configuration file and the `kubectl apply` command.

The ReplicaSet configuration file in Example 9-1 will ensure one copy of the `gcr.io/kuar-demo/kuard-amd64:green` container is running at any given time. Use the `kubectl apply` command to submit the `kuard` ReplicaSet to the Kubernetes API:

```
$ kubectl apply -f kuard-rs.yaml
replicaset "kuard" created
```

Once the `kuard` ReplicaSet has been accepted, the ReplicaSet controller will detect that there are no `kuard` Pods running that match the desired state and create a new `kuard` Pod based on the contents of the Pod template:

```
$ kubectl get pods
NAME       READY    STATUS    RESTARTS   AGE
kuard-yvzgd  1/1      Running   0          11s
```

Inspecting a ReplicaSet

As with Pods and other Kubernetes API objects, if you are interested in further details about a ReplicaSet, you can use the `describe` command to provide much more information about its state. Here is an example of using `describe` to obtain the details of the ReplicaSet we previously created:

```
$ kubectl describe rs kuard
Name:         kuard
Namespace:    default
Selector:     app=kuard,version=2
Labels:       app=kuard
              version=2
Annotations:  <none>
Replicas:     1 current / 1 desired
Pods Status:  1 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
```

You can see the label selector for the ReplicaSet, as well as the state of all of the replicas it manages.

Finding a ReplicaSet from a Pod

Sometimes you may wonder if a Pod is being managed by a ReplicaSet, and if it is, which one. To enable this kind of discovery, the ReplicaSet controller adds an `ownerReferences` section to every Pod that it creates. If you run the following, look for the `ownerReferences` section:

```
$ kubectl get pods <pod-name> -o=jsonpath='{.metadata.ownerReferences[0].name}'
```

If applicable, this will list the name of the ReplicaSet that is managing this Pod.

Finding a Set of Pods for a ReplicaSet

You can also determine the set of Pods managed by a ReplicaSet. First, get the set of labels using the `kubectl describe` command. In the previous example, the label selector was `app=kuard,version=2`. To find the Pods that match this selector, use the `--selector` flag or the shorthand `-l`:

```
$ kubectl get pods -l app=kuard,version=2
```

This is exactly the same query that the ReplicaSet executes to determine the current number of Pods.

Scaling ReplicaSets

You can scale ReplicaSets up or down by updating the `spec.replicas` key on the ReplicaSet object stored in Kubernetes. When you scale up a ReplicaSet, it submits new Pods to the Kubernetes API using the Pod template defined on the ReplicaSet.

Imperative Scaling with kubectl scale

The easiest way to achieve this is using the `scale` command in `kubectl`. For example, to scale up to four replicas, you could run:

```
$ kubectl scale replicasets kuard --replicas=4
```

While such imperative commands are useful for demonstrations and quick reactions to emergency situations (such as a sudden increase in load), it is important to also update any text file configurations to match the number of replicas that you set via the imperative `scale` command. The reason for this becomes obvious when you consider the following scenario.

Alice is on call, when suddenly there is a large increase in load on the service she is managing. Alice uses the `scale` command to increase the number of servers responding to requests to 10, and the situation is resolved. However, Alice forgets to update the ReplicaSet configurations checked into source control.

Several days later, Bob is preparing the weekly rollouts. Bob edits the ReplicaSet configurations stored in version control to use the new container image, but he doesn't notice that the number of replicas in the file is currently 5, not the 10 that Alice set in response to the increased load. Bob proceeds with the rollout, which both updates the container image and reduces the number of replicas by half. This causes an immediate overload, which leads to an outage.

This fictional case study illustrates the need to ensure that any imperative changes are immediately followed by a declarative change in source control. Indeed, if the need is not acute, we generally recommend only making declarative changes as described in the following section.

Declaratively Scaling with kubectl apply

In a declarative world, you make changes by editing the configuration file in version control and then applying those changes to your cluster. To scale the `kuard` ReplicaSet, edit the *kuard-rs.yaml* configuration file and set the `replicas` count to `3`:

```
...
spec:
  replicas: 3
...
```

In a multiuser setting, you would likely have a documented code review of this change and eventually check the changes into version control. Either way, you can then use the `kubectl apply` command to submit the updated `kuard` ReplicaSet to the API server:

```
$ kubectl apply -f kuard-rs.yaml
replicaset "kuard" configured
```

Now that the updated `kuard` ReplicaSet is in place, the ReplicaSet controller will detect that the number of desired Pods has changed and that it needs to take action to realize that desired state. If you used the imperative `scale` command in the previous section, the ReplicaSet controller will destroy one Pod to get the number to three. Otherwise, it will submit two new Pods to the Kubernetes API using the Pod template defined on the `kuard` ReplicaSet. Regardless, use the `kubectl get pods` command to list the running `kuard` Pods. You should see output similar to the following with three Pods in running state; two will have a smaller age because they were recently started:

```
$ kubectl get pods
NAME        READY    STATUS    RESTARTS   AGE
kuard-3a2sb   1/1      Running   0          26s
kuard-wuq9v   1/1      Running   0          26s
kuard-yvzgd   1/1      Running   0          2m
```

Autoscaling a ReplicaSet

While there will be times when you want to have explicit control over the number of replicas in a ReplicaSet, often you simply want to have "enough" replicas. The definition varies depending on the needs of the containers in the ReplicaSet. For example, with a web server like NGINX, you might want to scale due to CPU usage. For an in-memory cache, you might want to scale with memory consumption. In some cases, you might want to scale in response to custom application metrics. Kubernetes can handle all of these scenarios via *Horizontal Pod Autoscaling* (HPA).

"Horizontal Pod Autoscaling" is kind of a mouthful, and you might wonder why it is not simply called "autoscaling." Kubernetes makes a distinction between *horizontal* scaling, which involves creating additional replicas of a Pod, and *vertical* scaling, which involves increasing the resources required for a particular Pod (such as increasing the CPU required for the Pod). Many solutions also enable *cluster* autoscaling, where the number of machines in the cluster is scaled in response to resource needs, but that solution is outside the scope of this chapter.

NOTE

Autoscaling requires the presence of the `metrics-server` in your cluster. The `metrics-server` keeps track of metrics and provides an API for consuming metrics that HPA uses when making scaling decisions. Most installations of Kubernetes include `metrics-server` by default. You can validate its presence by listing the Pods in the `kube-system` namespace:

```
$ kubectl get pods --namespace=kube-system
```

You should see a Pod with a name that starts with `metrics-server` somewhere in that list. If you do not see it, autoscaling will not work correctly.

Scaling based on CPU usage is the most common use case for Pod autoscaling. You can also scale based on memory usage. CPU-based autoscaling is most useful for request-based systems that consume CPU proportionally to the number of requests they are receiving, while using a relatively static amount of memory.

To scale a ReplicaSet, you can run a command like the following:

```
$ kubectl autoscale rs kuard --min=2 --max=5 --cpu-percent=80
```

This command creates an autoscaler that scales between two and five replicas with a CPU threshold of 80%. To view, modify, or delete this resource, you can use the standard `kubectl` commands and the `horizontalpodautoscalers` resource. It is quite a bit to type `horizontalpodautoscalers`, but it can be shortened to `hpa`:

```
$ kubectl get hpa
```

WARNING

Because of the decoupled nature of Kubernetes, there is no direct link between the HPA and the ReplicaSet. While this is great for modularity and composition, it also enables some antipatterns. In particular, it's a bad idea to combine autoscaling with imperative or declarative management of the number of replicas. If both you and an autoscaler are attempting to modify the number of replicas, it's highly likely that you will clash, resulting in unexpected behavior.

Deleting ReplicaSets

When a ReplicaSet is no longer required, it can be deleted using the `kubectl delete` command. By default, this also deletes the Pods that are managed by the ReplicaSet:

```
$ kubectl delete rs kuard
replicaset "kuard" deleted
```

Running the `kubectl get pods` command shows that all the `kuard` Pods created by the `kuard` ReplicaSet have also been deleted:

```
$ kubectl get pods
```

If you don't want to delete the Pods that the ReplicaSet is managing, you can set the `--cascade` flag to `false` to ensure only the ReplicaSet object is deleted and not the Pods:

```
$ kubectl delete rs kuard --cascade=false
```

Summary

Composing Pods with ReplicaSets provides the foundation for building robust applications with automatic failover, and makes deploying those applications a breeze by enabling scalable and sane deployment patterns. Use ReplicaSets for any Pod you care about, even if it is a single Pod! Some people even default to using ReplicaSets instead of Pods. A typical cluster will have many ReplicaSets, so apply them liberally to the affected area.

Previous chapter

< 8. HTTP Load Balancing with Ingress

Next chapter

10. Deployments >